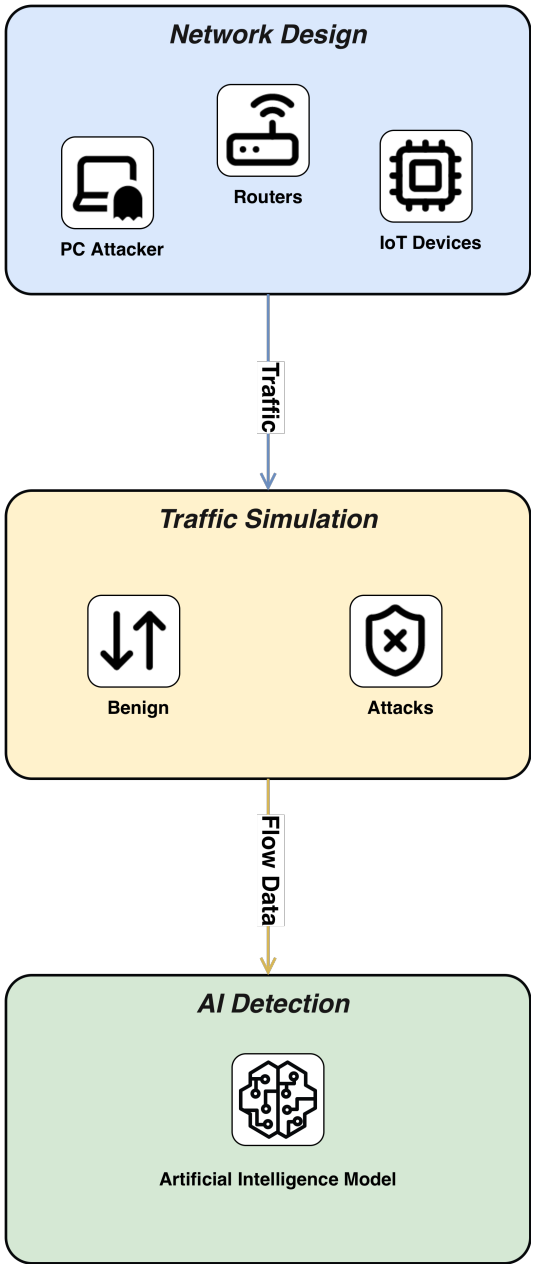
Graphical Abstract

IoT-Sim: An Interactive Platform for Designing and Securing Smart Device Networks

Alejandro Diez Bermejo, Branly Martinez Gonzalez, Beatriz Gil-Arroyo, Jaime Rincón Arango, Daniel Urda Muñoz



Highlights

IoT-Sim: An Interactive Platform for Designing and Securing Smart Device Networks

Alejandro Diez Bermejo, Branly Martinez Gonzalez, Beatriz Gil-Arroyo, Jaime Rincón Arango, Daniel Urda Muñoz

- Developed a lightweight, modular, and open-source tool designed to create, configure, and test attack detection models for Internet of Things (IoT) networks.
- It functions as a client-side web application developed in Angular, running entirely in the user's browser without needing a dedicated backend server.
- It was developed to address the lack of flexible simulation environments that allow researchers to replicate IoT scenarios and assess machine learning models in controlled, realistic conditions.
- Key functionalities include visually designing network topologies, simulating network traffic, injecting controlled attacks (such as Denial of Service (DoS)), and integrating trained AI models to analyze traffic and detect intrusions in real-time.
- The tool is designed for researchers and students, offering an accessible platform for benchmarking detection algorithms and facilitating reproducible experiments without the need for expensive or specialized hardware.

IoT-Sim: An Interactive Platform for Designing and Securing Smart Device Networks

Alejandro Diez Bermejo[a,*], Branly Martinez Gonzalez[a], Beatriz Gil-Arroyo[a], Jaime Rincón Arango[a], Daniel Urda Muñoz[a]

[a]*Grupo de Inteligencia Computacional Aplicada (GICAP), Departamento de Digitalización, Escuela Politécnica Superior. Universidad de Burgos. Av. Cantabria s/n. 09006 Burgos (Spain).*

Abstract

The IoT-Sim is a lightweight and modular tool designed to create, configure, and test models that detect attacks in Internet of Things (IoT) networks. It provides an interactive environment for simulating communication among connected devices and evaluating intrusion detection models. This framework allows researchers to design network topologies, inject different types of attacks, and benchmark detection algorithms under controlled conditions. By combining usability and flexibility in an open-source design, the simulator is a valuable resource for the education, research, and rapid prototyping of IoT security solutions.

Keywords: IoT Networks, Simulation Framework, Intrusion Detection, Attack Detection Models, Experimental Testbed, Machine Learning for Security, Models Benchmarking

Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v1.0.5
C2	Permanent link to code/repository used for this code version	`https://github.com/gicap-ubu/iot-sim`

*Corresponding author

Email addresses: `adbermejo@ubu.es` (Alejandro Diez Bermejo), `bamgonzalez@ubu.es` (Branly Martinez Gonzalez), `bgarroyo@ubu.es` (Beatriz Gil-Arroyo), `jarincon@ubu.es` (Jaime Rincón Arango), `durda@ubu.es` (Daniel Urda Muñoz)

C3	Permanent link to Reproducible Capsule	`https://gicap-ubu.github.io/iot-sim/`
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	TypeScript, HTML, CSS, Angular, TensorFlow
C7	Compilation requirements, operating environments & dependencies	Node (>= 20.19.0), Angular CLI (>= v21.0.0), Python (>= 3.10 < 3.12), TensorFlow (>= 2.15.0), tensorflowjs, Google Chrome (>= 113) | MS Edge (>= 113) | Safari (>= 16.4) | Mozilla Firefox (>= 113)
C8	If available Link to developer documentation/manual	`https://github.com/gicap-ubu/iot-sim/wiki`
C9	Support email for questions	`adbermejo@ubu.es`

Table 1: Code metadata

1. Motivation and significance

The rapid expansion of the Internet of Things (IoT) has resulted in a vast number of interconnected devices projected to reach approximately 40 billion worldwide by 2030 [1]. This unprecedented growth has generated highly heterogeneous network traffic and has created an increasingly complex and distributed communication ecosystem. This massive expansion has significantly increased both data generation and the potential attack surface, making IoT networks particularly vulnerable to a variety of cyber threats [2]. According to Forescout's Riskiest Connected Devices of the 2025 report, routers alone account for more than 50% of devices with critical vulnerabilities, surpassing computers and wireless access points [3]. These findings underline the increasing exposure of IoT infrastructure to cyberattacks that target the network layer.

Despite the abundance of research on Intrusion Detection Systems (IDS) [4, 5], there is still a lack of flexible, open-source simulation environments that allow researchers to reproduce IoT network scenarios and evaluate machine learning models under realistic, controlled conditions [6]. The IoT-Sim was developed to address this gap by providing a platform where users can design network topologies, generate traffic, and simulate cyberattacks while integrating artificial intelligence models for anomaly detection.

The simulator contributes to scientific discovery by enabling reproducible ex-

periments in complex IoT environments and providing a benchmark platform for evaluating and comparing different machine learning algorithms, hyperparameters, and detection strategies. Users can interact with the simulator through a graphical interface, allowing them to configure IoT topologies, generate realistic network traffic, inject attacks such as Denial of Service, and evaluate models on the simulated data.
The simulator is designed to provide a comprehensive environment for IoT network research. Unlike network simulators such as NEMO [7], NS-3 [8] or CORE [9] (compared in Table 2), which focus primarily on detailed network modeling and offer limited integration with machine learning for cybersecurity, this simulator enables users to evaluate a wide range of artificial intelligence algorithms and models on the generated traffic. Integrating IoT network simulation, attack modeling, and machine learning evaluation within a single reproducible framework offers a complete platform for studying, benchmarking, and improving intrusion detection strategies in IoT networks.

Table 2: Comparison of IoT-Sim with existing network simulation tools.

Feature	IoT-Sim (Proposed)	NEMO [7]	NS-3 [8]	CORE [9]
Architecture	Client-side Web (Angular)	Emulator (Java)	Discrete Event (C++/Python)	Emulator (Linux Namespaces)
Prerequisites	Modern Web Browser	Java VM (JRE 1.8+)	Compiler Toolchain	Linux Kernel
Setup Complexity	None (Zero-install)	Low	High	Medium
AI/ML Integration	Native (TensorFlow.js)	Manual	External (Modules)	External
User Interface	Interactive GUI	CLI	Scripts (C++) & NetAnim	GUI/CLI
Target Audience	Education & Prototyping	Scalable Emulation	Protocol Development	Network Ops & Testing
Scalability	Medium (Browser-bound)	High	High	Medium

2. Software description

The IoT-Sim is a web application developed to simulate IoT networks and evaluate intrusion detection models in a controlled environment. It was conceived as a research-oriented platform to enable the testing of cybersecurity algorithms without requiring costly or large-scale IoT infrastructure.
The development followed an incremental methodology based on Scrum [10] and applied object-oriented programming principles [11], separation of concerns, and established design patterns such as Singleton, Observer, and component-based architectures [12].
The simulator enables users to design and configure network topologies, simulate data flows, execute controlled attacks, and analyze traffic using AI-based detection models. Its functionality includes the main areas of network design, flow simulation, attack execution, and intelligent attack detection.
The target users are researchers and students. The design prioritizes usability and accessibility through a clear interface, minimal interaction steps, immediate visual feedback, and compatibility with modern operating systems.

All functionalities are documented in detail and presented within an interactive environment, enabling safe experimentation, validation of detection models, and extension of the simulator to custom attacks or models.

2.1. Software architecture

The IoT-Sim was designed as a client-side application that was developed entirely in Angular [13]. Its architecture follows a layered and modular design that promotes the separation of concerns, maintainability, and extensibility. All processes, from network simulation to attack generation and visualization, are executed locally in the browser, without the need for a dedicated backend server (see Fig. 1).

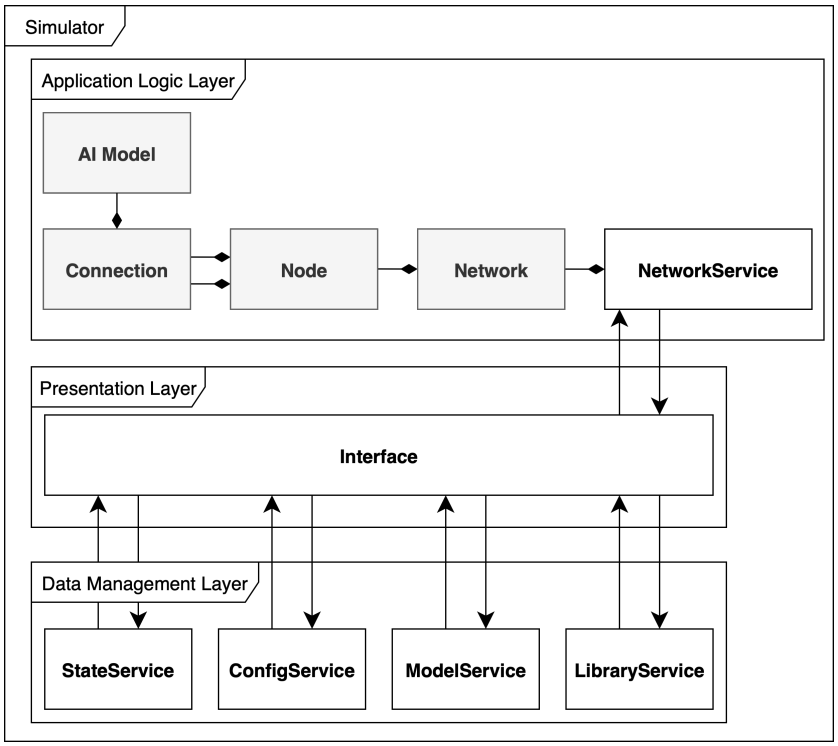


Figure 1: General architecture of the IoT-Sim.

The system is organized into three main layers:

- **Presentation Layer:** This layer manages user interaction through a graphical interface. It allows users to design IoT network topologies, configure nodes and connections, launch attacks, and visualize traffic and detection results. The interface was built using Angular components and styled with Tailwind CSS [14] and Spartan UI [15] to ensure responsiveness and usability.

- **Application Logic Layer:** This layer contains the simulation engine, responsible for modeling network behavior based on the OSI model [16]. It defines nodes, routers, packets, and connections as part of a network abstraction implemented through an object-oriented design, coordinating how data flow between entities. It also manages the execution of external scripts, allowing users to import custom attacks, commands, and interception modules. Furthermore, this layer executes AI-based detection models: it receives network flow data, applies trained models in real time to detect malicious activity or anomalies, and forwards the results for visualization and analysis.

- **Data Management Layer:** This layer manages configuration files, simulation states, and records of network flows. It is also responsible for storing and loading trained AI models and related artifacts. In addition, it processes the command and attack libraries, handling ingestion, validation, and normalization of those modules, and enables the dynamic loading of new commands/attacks for use in simulations and model evaluation. Data are stored and exchanged using lightweight formats, allowing users to save and easily load network configurations, analysis results, and models.

The proposed client-side architecture inherently enhances execution safety by reducing the risks linked to running external scripts. Operating entirely within the browser's sandbox, the simulation runtime ensures that custom scripts remain isolated from the host's operating system and file system. Additionally, the lack of a backend server removes potential server-side injection points or cross-user contamination. To avoid runtime errors or instability caused by malformed inputs, the Data Management Layer enforces structural validation and normalization on all imported modules before they are instantiated in the Application Logic Layer.

2.2. Software functionalities

The IoT-Sim was designed as an experimental environment for testing and evaluating attack detection models in IoT networks. Its functionalities focus on providing realistic network behavior, controlled attack execution, and integration of AI-based detection models. The main functionalities are summarized as follows:

2.2.1. Network Design

Users can create and configure IoT network topologies by adding, editing, connecting, or removing nodes. The simulator allows saving and importing

network configurations to facilitate experimentation and reuse [1].

2.2.2. Configurable Network Conditions

The simulator supports the introduction of network variability such as latency or bandwidth limitations, allowing the evaluation of detection models under realistic network conditions.

2.2.3. Flow Simulation and Attack Generation

The system enables the simulation of network traffic between nodes and the execution of controlled attacks through a modular packet generation engine. Unlike simple traffic generators, this engine operates by programmatically constructs discrete packet objects that conform to standard Transport and Application layer protocol specifications [17]. In particular, the system implements granular control over protocol headers defined in the TCP/IP stack, allowing for the precise manipulation of fields such as TCP flags, sequence numbers, and ICMP type codes. By instantiating these packet models with customizable payloads and timing attributes, the simulator can faithfully reproduce complex network interactions, including the TCP three-way handshake, as well as generate protocol-specific anomalies required for security testing. In addition, users can import external scripts containing new implementations of commands or attack types, thereby extending the simulator's functionality [2]. This feature allows the generation of data that represent both normal and malicious behavior, which is essential for evaluating and testing attack-detection models.

2.2.4. Attack Detection Analysis

One of the main functionalities is the analysis of traffic flows using imported AI models. Users can load their own trained detection models developed with TensorFlow[3] [19], assign them to specific network connections, and visualize the real-time predictions of possible intrusions. This feature provides a practical method to compare and validate different models under identical network conditions.

2.3. Scalability and performance analysis

While the client-side architecture of IoT-Sim facilitates accessibility and zero-configuration deployment, it inherently binds simulation performance to the

[1] Examples of predefined schemas are provided in Appendix A

[2] Examples of external scripts are provided in Appendix A

[3] TensorFlow models must be converted via the tensorflowjs [18] tool and require a JavaScript script to execute the model. Examples are provided in Appendix A, and the process is detailed in the documentation.

host system's resources. In contrast to server-side simulators capable of horizontal scaling across clusters, IoT-Sim operates within the strict boundaries of the browser sandbox. To quantify these constraints, we evaluated the system on a standard consumer workstation (Chromium v.144). The empirical results are summarized in Table 3.
This architecture introduces specific constraints and optimization strategies:

- **Memory Usage and Scalability Limits:** The simulation state is maintained in the browser's memory heap, causing resource usage to grow linearly with the number of nodes and active packet flows. As shown in Table 3, the system maintains stable performance with negligible latency for small topologies (10-20 nodes). However, approaching 50 nodes, the memory footprint increases significantly ($\sim$2.6 GB) due to the overhead of DOM rendering and the simultaneous execution of AI models, introducing noticeable UI latency.

- **Computational Overhead and AI Optimization:** The execution of AI detection models is the most computationally intensive task, evidenced by the CPU usage spike at 50 nodes ($\sim$180%, utilizing multiple cores). To address this, IoT-Sim implements WebGL acceleration via TensorFlow.js, offloading heavy matrix computations to the GPU. This strategy prevents the main thread from blocking completely, keeping the application responsive even under high load.

- **Future Optimization Strategies:** To further mitigate browser resource constraints in large-scale scenarios, the modular design allows for future integration of Web Workers (to offload packet generation from the UI thread) and WebAssembly modules (to execute core simulation loops at near-native speed).

Consequently, the current architecture prioritizes functional scalability over massive quantitative scale. The framework efficiently addresses the computational demands of protocol logic and AI execution within limited-scale topologies. Consequently, it operates not as a substitute, but as a flexible auxiliary to rigorous distributed emulators, specifically targeting the requirements of academic prototyping and educational scenarios.

3. Illustrative examples

Figure 2 shows a screenshot of the general view of the IoT-Sim running a Denial of Service (DoS) attack [20] on a simple network composed of four nodes and one router.

Table 3: System performance metrics across varying node counts with concurrent AI inference and continuous network flows (Chromium v.144).

Nodes	CPU Usage	RAM Usage	UI Latency	Status
10	$\sim$ 32%	$\sim$ 387 MB	Negligible	Stable
20	$\sim$ 40%	$\sim$ 510 MB	Low	Stable
50	$\sim$ 180%	$\sim$ 2.6 GB	Medium	Laggy

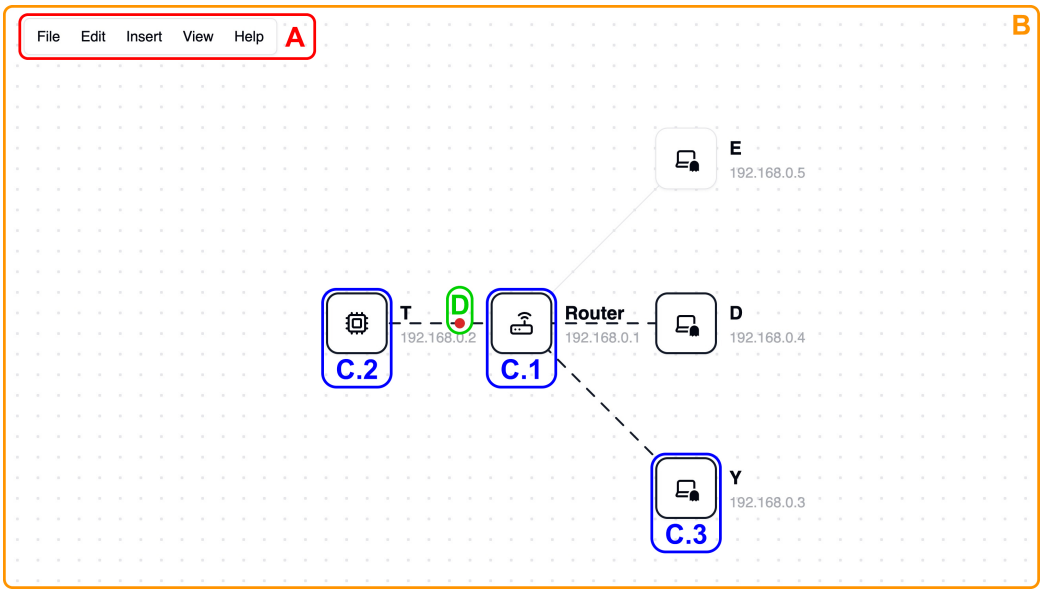


Figure 2: The IoT-Sim showing the detection of a network attack. The graphical user interface components are highlighted with different colors and labeled A–D. Descriptions of these components are provided in the text.

The simulator window is divided into several areas. The main area, (B), is a canvas in which objects can be added to design the network topology. The topology comprises two types of elements:

- **Nodes:** Shown in the figure as (C), nodes are the primary elements of the network architecture. Nodes can be of different types, depending on their functions:
  - **Router (C.1):** The network's core component; this device enables communication between other nodes.
  - **IoT device (C.2):** Typical network endpoints that may be targeted by attacks.
  - **Computer (C.3):** Devices that can launch attacks against other nodes.

- **Connections:** Connect a node to the router, enabling communication with all networks. The connection allows the analysis of all network traffic passing through it. When a detection model is active on the connection, a colored circle indicates the model's status, as shown in the figure as (D). The status can be one of the following: processing data (blue), attack detected (red), or normal traffic (green).

The application menu (A) allows users to control all main configurations of the simulator. It provides options for opening existing projects, saving the current one, loading custom libraries containing commands and attacks, and importing detection models. In addition, it enables users to undo modifications, insert new nodes, and adjust visual settings to improve the accessibility and usability of the simulation environment.
In addition to the main view, the IoT-Sim provides several complementary views that allow users to interact with specific network components and configure the simulation parameters in detail. Figures 3, 4, 5, and 6 illustrates these additional views, which are displayed when a user clicks on a specific node or connection.

- **Node traffic and command panel (Fig. 3):** Shows the incoming and outgoing traffic of a selected node. It also provides a section for executing custom commands imported by the user, thereby enabling the generation of simulated benign traffic. This unified view simplifies the verification process, allowing users to instantly confirm that injected commands are generating the expected traffic patterns.

- **Attack configuration panel (Fig. 4):** Enables the setup and customization of simulated attacks. Users can select the attack type, configure their parameters, and assign target devices within the network. By presenting complex scripts through a simple interface, the tool lowers the technical barrier for security testing and facilitates rapid experimentation with different attack scenarios.

- **Node settings panel (Fig. 5):** Provides options for modifying the properties of individual nodes, such as the IP address, name, and type. This panel facilitates quick adjustments to the network topology, allowing researchers to agilely prototype and dynamically adjust node roles without resetting the entire simulation environment.

- **Connection and detection model panel (Fig. 6):** Displays detailed information regarding the connections between nodes and routers. It also allows users to attach and manage detection models associated

with each connection and visualize their operational status. This mechanism simplifies model deployment, providing immediate visual feedback that helps users interpret detection performance and sensitivity in real time.

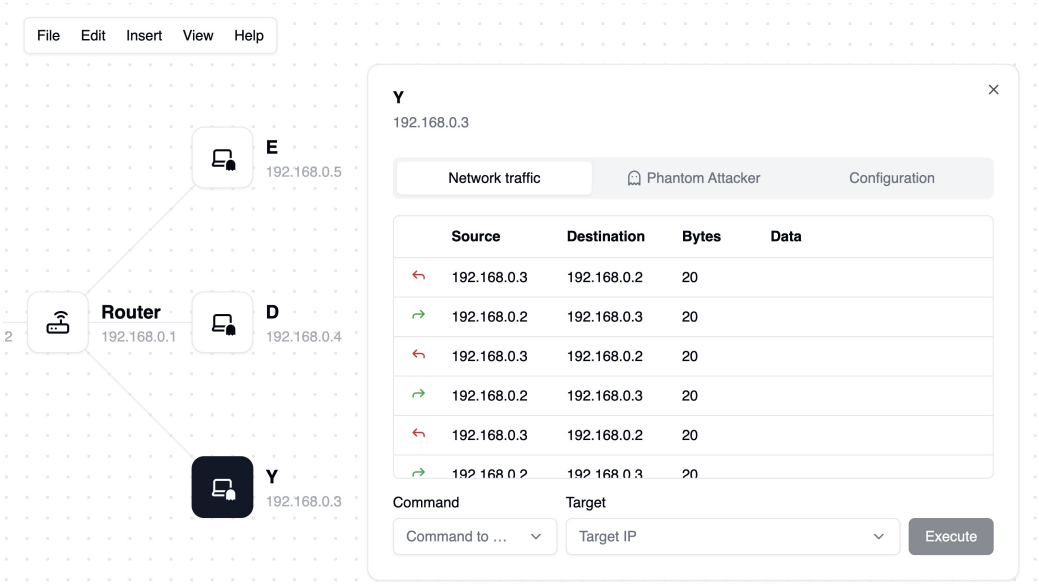


Figure 3: Node traffic and command panel.

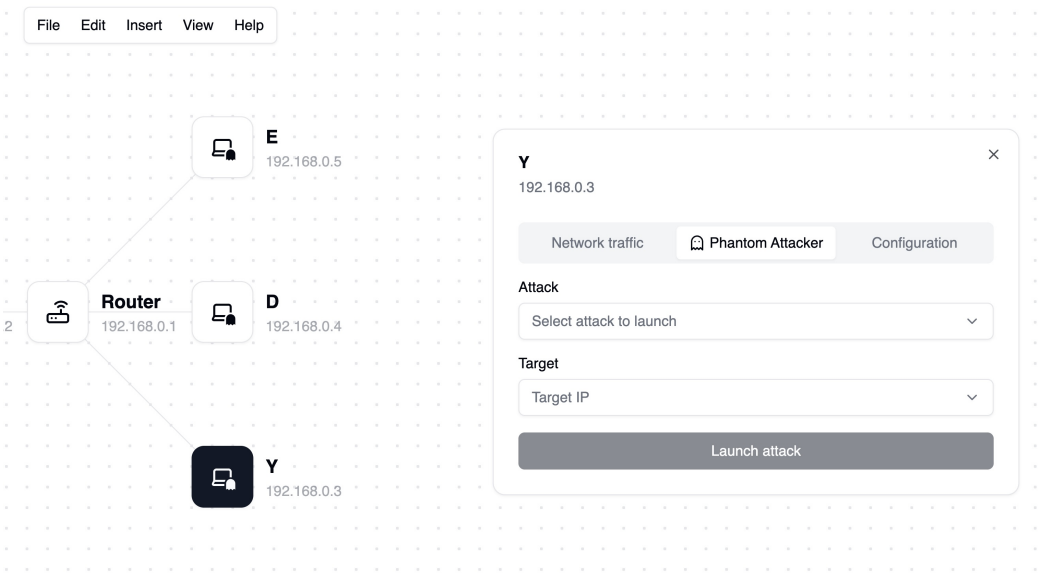


Figure 4: Attack configuration panel.

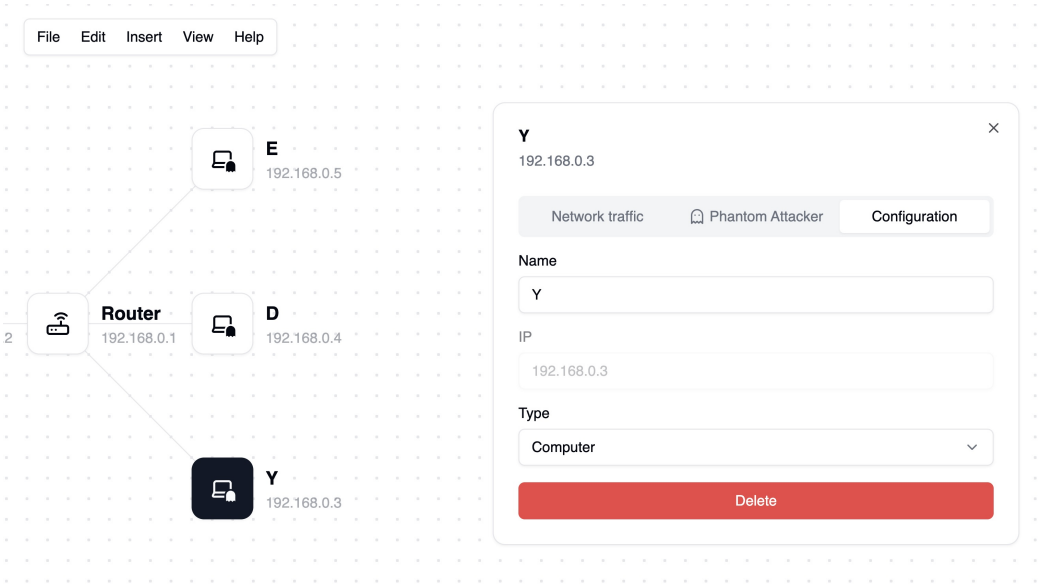


Figure 5: Node settings panel.

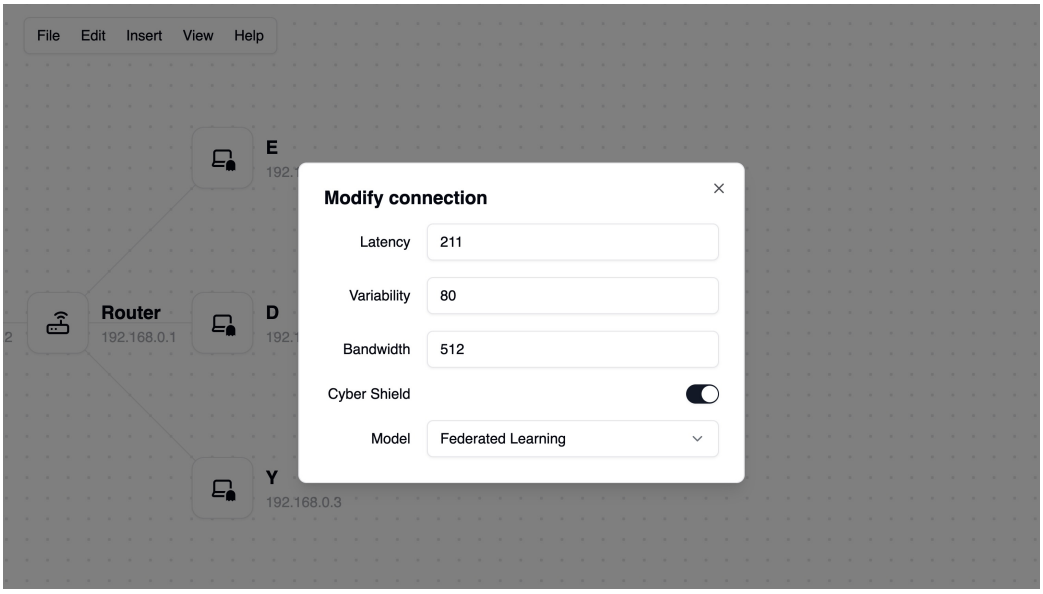


Figure 6: Connection and detection model panel.

4. Impact

The IoT-Sim provides an open and modular experimental environment for studying and evaluating attack detection models in IoT networks. Its client-side implementation makes it easily accessible and extensible, lowering the

barrier to reproducing IoT scenarios and testing AI-based intrusion detection systems without the need for specialized hardware or complex deployments. The simulator was used in a controlled laboratory environment to evaluate two attack detection models trained under different paradigms, centralized and federated learning [21], demonstrating its suitability for comparing heterogeneous training strategies under identical network conditions. By enabling reproducible and configurable experiments, it facilitates the systematic analysis of model robustness, adaptability, and performance in diverse attack scenarios.

Beyond these initial experiments, the IoT-Sim opens up new research directions related to lightweight and distributed detection mechanisms for resource-constrained IoT devices. It also allows researchers to analyze model behavior under different network configurations and attack conditions, thereby supporting the evaluation of detection performance in diverse IoT environments.

Although its current use remains within the research laboratory, the simulator holds strong potential for broader adoption in academic and industrial contexts. Future developments will focus on expanding the network simulation capabilities and incorporating new communication protocols, traffic models, and attack behaviors to achieve more realistic and versatile experimentation environments for intelligent IoT network management.

5. Conclusions

The IoT-Sim is a practical, lightweight, and flexible platform for designing, configuring, and exploring IoT network topologies while testing attack-detection models under controlled and reproducible conditions. Its modular client-side architecture supports rapid prototyping and experimentation without the need for a specialized infrastructure, allowing researchers and students to generate realistic traffic, inject diverse attack behaviors, and observe model responses in real time.

The integrated support for importing external scripts and TensorFlow models facilitates a direct comparison of heterogeneous detection approaches within identical simulated environments. This capability promotes rigorous benchmarking of algorithms, hyperparameter settings, and deployment strategies and strengthens the reproducibility of experimental results by allowing the same scenarios and artifacts to be shared and re-executed.

The simulator focuses on usability and configurability, reducing the entry barrier for security experimentation in resource-constrained IoT environments, and promotes iterative exploration of model robustness across variable network conditions, such as latency and bandwidth limitations. The visual

component-based interface and the per-connection model attachment mechanism facilitate the precise analysis of detection performance and enable investigations into localized versus global detection strategies.
The tool's current implementation and demonstrated use in laboratory comparisons expose research opportunities for lightweight distributed detection, model adaptation to changing traffic patterns, and privacy-preserving training workflows for IoT ecosystems. The simulator creates a controlled testbed for studying resilience to evasion techniques, sensitivity to attack parameterization, and trade-offs between detection accuracy, latency, and computational overhead on constrained devices.
Future development directions will enhance realism and extend applicability by incorporating additional communication protocols, richer traffic generators, and support for longitudinal and large-scale scenario orchestration. These enhancements will increase the fidelity of experiments, broaden the range of empirical questions that can be addressed, and strengthen the simulator's role as a reference platform for the design and evaluation of intelligent, deployable IoT security solutions.
Overall, the IoT-Sim constitutes a valuable contribution to the research infrastructure for IoT cybersecurity by combining accessibility, modularity, and machine learning integration to accelerate reproducible experimentation, methodological comparison, and the development of more robust intrusion detection strategies for heterogeneous IoT environments.

CRediT authorship contribution statement

Alejandro Diez Bermejo: Writing – original draft, Software, Visualization, Conceptualization, Methodology, Investigation, Validation.
Branly Martinez Gonzalez: Writing – review and editing, Data curation, Investigation.
Beatriz Gil Arroyo: Writing – review and editing.
Jaime Rincón Arango: Writing – review and editing, Supervision, Conceptualization.
Daniel Urda Muñoz: Writing – review and editing, Supervision, Conceptualization, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used ChatGPT to proofread and improve the language. After using this tool/service, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

This publication is part of the AI4SECIoT project ("Artificial Intelligence for Securing IoT Devices"), funded by the National Cibersecurity Institute (INCIBE), derived from a collaboration agreement signed between the National Institute of Cybersecurity (INCIBE) and the University of Burgos. This initiative is carried out within the framework of the Recovery, Transformation and Resilience Plan funds, financed by the European Union (Next Generation), the project of the Government of Spain that outlines the roadmap for the modernization of the Spanish economy, the recovery of economic growth and job creation, for solid, inclusive and resilient economic reconstruction after the COVID19 crisis, and to respond to the challenges of the next decade.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at `https://gicap-ubu.github.io/iot-sim/resources/`.

References

[1] S. Sinha, State of IoT 2024: Number of connected IoT devices growing 13% to 18.8 billion globally, `https://iot-analytics.com/number-connected-iot-devices`, [cited 2025-10-22] (2024).

[2] M. Mahajan, K. Bais, D. M. Kotambkar, K. Paroha, Security for the internet of things: A comprehensive review and analysis, in: 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), 2024, pp. 1–7. `doi:10.1109/ICCCNT61001.2024.10724790`.

[3] Vedere Research Labs, The riskiest connected devices in 2025, `https://www.forescout.com/resources/riskiest-devices-2025-report/`, [cited 2025-10-22] (2025).

[4] IBM, What is an Intrusion Detection System (IDS)? | IBM, [cited 2025-10-26] (Apr. 2023).
URL https://www.ibm.com/think/topics/intrusion-detection-system

[5] M. A. Al-Garadi, A. Mohamed, A. K. Al-Ali, X. Du, I. Ali, M. Guizani, A survey of machine and deep learning methods for internet of things (iot) security, IEEE Communications Surveys & Tutorials 22 (3) (2020) 1646–1685. doi:10.1109/COMST.2020.2988293.

[6] R. Almutairi, G. Bergami, G. Morgan, Advancements and challenges in iot simulators: A comprehensive review, Sensors 24 (5) (2024). doi:10.3390/s24051511.
URL https://www.mdpi.com/1424-8220/24/5/1511

[7] L. Veltri, L. Davoli, R. Pecori, A. Vannucci, F. Zanichelli, Nemo: A flexible and highly scalable network emulator, SoftwareX 10 (2019) 100248. doi:https://doi.org/10.1016/j.softx.2019.100248.
URL https://www.sciencedirect.com/science/article/pii/S2352711019300135

[8] ns-3, ns-3: A discrete-event network simulator for internet systems, https://www.nsnam.org/, [cited 2025-10-22] (2008).

[9] Naval Research Laboratory, CORE (common open research emulator), https://www.nrl.navy.mil/Our-Work/Areas-of-Research/Information-Technology/NCS/CORE/, [cited 2025-10-22] (2008).

[10] M. Layton, S. Ostermiller, D. Kynaston, Agile Project Management For Dummies, 3rd Edition, John Wiley & Sons, Hoboken, NJ, 2020.

[11] M. Weisfeld, The Object-Oriented Thought Process, 5th Edition, Addison-Wesley Professional, Boston, 2019.

[12] A. Shalloway, J. R. Trott, Design Patterns Explained: A New Perspective on Object-Oriented Design, 2nd Edition, Addison-Wesley, Boston, 2004.

[13] Angular, Angular: The framework for building scalable web apps with confidence, https://angular.dev, [cited 2025-10-15] (2016).

[14] Tailwind Labs, Tailwind CSS: Rapidly build modern websites without ever leaving your HTML, https://tailwindcss.com, [cited 2025-10-15] (2017).

[15] Goetzrobin, Spartan: Cutting-edge tools powering Angular full-stack development, https://spartan.ng, [cited 2025-10-15] (2023).

[16] IBM, What Is the OSI Model? | IBM, [cited 2025-11-01] (Jun. 2024).
URL https://www.ibm.com/think/topics/osi-model

[17] W. R. Stevens, TCP/IP illustrated (vol. 1): the protocols, Addison-Wesley Longman Publishing Co., Inc., USA, 1993.

[18] TensorFlow, Tensorflow.js converter, https://github.com/tensorflow/tfjs-converter/blob/master/tfjs-converter/README.md, gitHub Repository, [cited 2025-10-15] (2018).

[19] TensorFlow, Tensorflow: Large-scale machine learning on heterogeneous systems, https://www.tensorflow.org, [cited 2025-10-15] (2015).

[20] Cloudflare, How to DDoS | DoS and DDoS attack tools, https://www.cloudflare.com/learning/ddos/ddos-attack-tools/how-to-ddos/, [cited 2025-11-05].

[21] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data (2023). arXiv:1602.05629.
URL https://arxiv.org/abs/1602.05629